Offer Show 微信公众号后台系统
文档三：系统详细设计文档

顾远　23301095

2026 年 4 月 27 日

目录

1 文档概述 **3**
1.1 编写目的 3
1.2 设计目标 3

2 系统总体架构 **3**
2.1 技术选型 3
2.2 逻辑架构 4
2.3 后端工程结构（FastAPI） 5
2.4 部署拓扑 5
2.5 微信公众号交互流程 5

3 领域模型与数据库设计 **6**
3.1 ER 总览 6
3.2 核心数据表 6
3.2.1 user 6
3.2.2 job_posting 6
3.2.3 salary_report 7
3.2.4 其他表（简述） 8

4 API 设计总则 **8**
4.1 基础约定 8
4.2 标准方法映射 8
4.3 分页约定 9
4.4 统一响应结构 9
4.5 错误码示例 9
4.6 权限控制 9

5 对外 API 详细设计 **10**
5.1 认证与用户 . . . 10
5.1.1 微信登录 . . . 10
5.1.2 获取当前用户 . . . 10
5.1.3 用户管理（管理员） . . . 10
5.2 公司（Company） . . . 10
5.3 招聘信息（JobPosting）——核心 . . . 11
5.3.1 List 招聘信息 . . . 11
5.3.2 Get / Create / Update / Replace / Delete . . . 11
5.3.3 批量上传招聘信息 . . . 12
5.3.4 招聘岗位（Position）子资源 . . . 12
5.4 字典（Industry / City / Batch） . . . 12
5.5 求职早报（DailyBrief）与订阅 . . . 12
5.6 求职好课（Course） . . . 13
5.7 求职专栏 / 薪资专栏（Column） . . . 13
5.8 内推集合 / 社招集合 . . . 13
5.9 实习点评（Review） . . . 13
5.10 顶尖人才计划（EliteProgram） . . . 13
5.11 薪资爆料（SalaryReport）——核心 . . . 13
5.11.1 List 薪资爆料 . . . 13
5.11.2 Get / Create / Update / Replace / Delete . . . 14
5.11.3 批量导入薪资爆料 . . . 15
5.11.4 薪资人气榜 . . . 15
5.11.5 校招 / 社招视图 . . . 15
5.11.6 薪资爆料评论（SalaryComment） . . . 15
5.12 统一搜索 . . . 15

6 关键能力实现要点 **15**
6.1 部分检索（关键字） . . . 15
6.2 批处理 . . . 16
6.3 分页 . . . 16
6.4 权限控制与反作弊 . . . 16
6.5 微信公众号能力对接（WeChatPy） . . . 16
6.6 前端（Vue 3 + Vant 4）要点 . . . 17

7 需求与作业要求映射 **17**

8 后续工作 **17**

1 文档概述

1.1 编写目的

本文档基于《文档二：需求文档》中对 Offer Show 小程序” 找名企 → 名企实习校招” 与” 查薪资” 两大业务模块的需求分析，给出后台系统的总体架构、模块划分、领域模型、数据库设计与对外 API 设计。本文档是后续编码实现（文档四：系统代码）的输入。

1.2 设计目标

- 完整覆盖需求文档中的全部功能场景；
- 提供标准的 RESTful API（包含 `get` / `list` / `create` / `update` / `replace` / `delete` 六类标准方法）；
- 支持关键字部分检索、批处理、分页、基于 RBAC 的权限控制；
- 易于通过微信公众号 / 移动 H5 / Web 端复用；前端采用 Vue 3 + Vant 4 H5。

2 系统总体架构

2.1 技术选型

本项目为单人课程作业，强调” 快速落地、易演示、易部署”，最终选型如下表所示：

层次	选型	理由
前端 H5（移动版）	Vue 3 + Vant 4 + Vite + Pinia + axios	Vant 4 是面向移动端 H5 的组件库，Tab / List / PullRefresh / Picker 等可直接拼出 Offer Show 风格界面；Vite 构建快。
后端 Web 框架	Python 3.11 + FastAPI	异步、类型注解、原生 OpenAPI（Swagger UI / ReDoc）；单人开发效率高。
ORM / 迁移	SQLAlchemy 2.0 + Alembic	类型友好的 2.0 风格语法，Alembic 管理 schema 演进。
鉴权	OAuth2PasswordBearer + JWT（python-jose）	与 FastAPI 原生集成，Swagger UI 自带"Authorize" 按钮便于演示。
分页	fastapi-pagination	支持 `Page` 与 `LimitOffsetPage` 两种风格，改动极小。
校验	Pydantic v2	与 FastAPI 同源；自动产生 OpenAPI schema。
数据库	MySQL 8.0（utf8mb4_0900_ai_ci）	关系清晰，原生 `FULLTEXT` 满足关键字检索。
缓存 / 限流	Redis 7	限流（slowapi）、JWT 黑名单、排行榜 ZSET、热门列表缓存。
检索	MySQL FULLTEXT(ngram 分词器)	一体化部署、零依赖；后期可平滑替换为 Elasticsearch。
API 文档	FastAPI 内置（`/docs`、`/redoc`）	自动从 Pydantic 生成，演示与对接零成本。
反向代理	Nginx	静态资源托管 + HTTPS 终端 + 反代 `/api/`。
部署	Docker Compose	一条命令启动 MySQL / Redis / 后端 / Nginx。
微信对接	WeChatPy + 公众平台测试号	公众号消息回调 / 模板消息推送（早报）。

2.2 逻辑架构

系统按职责划分为三层：

- **接入层**：移动 H5（微信内 / 普通浏览器均可）、微信公众号消息回调、运营管理页面，统一经 Nginx 反向代理至后端。
- **应用层（FastAPI 单体）**：按业务模块组织 `routers/`，包括 `auth`、`users`、`companies`、`job_postings`、`positions`、`reviews`、`daily_briefs`、`subscriptions`、`courses`、`columns`、`elite_programs`、`salary_reports`、`salary_comments`、`search`、`wechat`；公共能力放 `core/`（鉴权、依赖注入、异常处理、限流中间件、风控）。
- **数据层**：MySQL 8.0（业务数据 + FULLTEXT 索引）、Redis 7（缓存 / 限流 / 黑名单 / 排行榜热度）、本地或对象存储（公司 LOGO、课程封面，开发期可用本地 `static/` 目录）。

2.3 后端工程结构（FastAPI）

建议采用如下目录组织：

```
offershow-backend/
    app/
        main.py              # FastAPI 实例、路由注册、CORS、限流
        core/
            config.py        # pydantic-settings
            security.py      # JWT 编解码 + OAuth2PasswordBearer
            deps.py          # get_db / get_current_user / require_role
            pagination.py    # fastapi-pagination 配置
            ratelimit.py     # slowapi 限流
            exceptions.py    # 全局异常 -> 统一响应
        db/
            session.py       # async engine / SessionLocal
            base.py          # DeclarativeBase
        models/              # SQLAlchemy 2.0 ORM
        schemas/             # Pydantic v2
        crud/                # 数据访问层
        routers/             # 各业务路由
        services/            # 复杂业务（搜索、风控、批处理）
        wechat/              # 公众号消息回调
    alembic/
    tests/
    docker-compose.yml
    pyproject.toml
```

2.4 部署拓扑

- `docker-compose.yml` 内编排 4 个服务：`mysql:8.0`、`redis:7-alpine`、`backend`（FastAPI + Uvicorn/Gunicorn）、`nginx`。
- Nginx：`/` 托管 Vue 打包后的静态文件（`dist/`），`/api/` 反代到 `backend:8000`，`/wechat/` 反代到公众号回调路径，`/docs`, `/redoc`, `/openapi.json` 透传。
- 后端启动命令：`gunicorn app.main:app -k uvicorn.workers.UvicornWorker -w 4`。
- 异步任务（早报定时推送、批量审核回调）使用 FastAPI 内置 `BackgroundTasks` 或 APScheduler；规模扩大后可平滑替换为 Celery + Redis broker。

2.5 微信公众号交互流程

1. 用户在公众号内点击菜单 / 输入关键字 → 公众平台推送 XML 到 `POST /wechat/callback`（WeChatPy 解析）。
2. H5 页内：用户首次进入调用 `POST /api/v1/auth/wx-login`（用 `code` 换 OpenID）；服务端颁发 JWT 并缓存进 Redis（TTL 2h，可作为黑名单/会话校验依据）。

3. 业务请求统一携带 `Authorization: Bearer <jwt>`，由 FastAPI 依赖 `get_current_user` 解析。
4. 早报推送：APScheduler 每日 09:00 触发，对订阅用户调用模板消息接口；失败重试入 Redis 队列。

3 领域模型与数据库设计

3.1 ER 总览

核心实体：`User`、`Company`、`JobPosting`、`Position`、`Subscription`、`Course`、`Column`、`Review`、`EliteProgram`、`SalaryReport`、`SalaryColumn`、`SalaryComment`、`Industry`、`City`、`Batch`（字典）。

3.2 核心数据表

3.2.1 user

字段	类型	说明
id	BIGINT PK	主键
openid	VARCHAR(64) UQ	微信 OpenID
nickname	VARCHAR(64)	昵称
avatar	VARCHAR(255)	头像
role	ENUM	guest / user / member / hr / admin
company_id	BIGINT	HR 角色绑定的公司
is_blocked	TINYINT	是否被封禁
risk_score	INT	反作弊风险分（0–100）
created_at, updated_at	DATETIME	

3.2.2 job_posting

字段	类型	说明
id	BIGINT PK	
company_id	BIGINT FK	关联 company
title	VARCHAR(128)	招聘标题
batch	VARCHAR(32)	招聘批次
recruitment_type	ENUM	校招/实习/社招/暑期实习/...
industry_code	VARCHAR(32)	关联 industry 字典
cities	JSON	工作城市数组

deliver_start, deliver_end	DATE	投递起止
graduation_range	VARCHAR(64)	招聘届属
apply_url	VARCHAR(255)	投递链接
internal_code	VARCHAR(32)	内推码
benefits	JSON	福利标签数组
description	MEDIUMTEXT	招聘简介
views, interest_count	INT	浏览数 / 感兴趣数
status	ENUM	draft / pending / online / offline
created_by	BIGINT	创建者 user_id
created_at, updated_at	DATETIME	

索引：`(company_id)`、`(recruitment_type, status, created_at)`、`(industry_code)`、ES 全文索引 `(title, company_name)`。

3.2.3 salary_report

字段	类型	说明
id	BIGINT PK	
company_id	BIGINT FK	
position	VARCHAR(64)	岗位名
city	VARCHAR(32)	城市
salary_desc	VARCHAR(64)	薪资描述（如 16*16）
annual_min, annual_max	DECIMAL(10,2)	年薪下/上限（万元）
recruitment_type	ENUM	校招/实习/社招
education	ENUM	大专/本科/硕士/博士
edu_tags	JSON	985/211/海归等
industry_code	VARCHAR(32)	
tags	JSON	# 年终奖 # 签字费 # 工作作息等
remark	TEXT	备注
credibility	TINYINT	可信度 0–100
views, likes	INT	
status	ENUM	pending / online / hidden
author_id	BIGINT	提交者
created_at, updated_at	DATETIME	

索引:(company_id, recruitment_type)、(status, created_at)、ES 索引 (company_name, position, city)。

3.2.4 其他表（简述）

- company(id, name, logo, industry_code, intro, ...)
- position(id, posting_id, name, description)
- review(id, company_id, user_id, score, content, status, created_at)
- subscription(id, user_id, type[早报/专栏], target_id, status, created_at)
- course(id, title, cover, category, price, url, status)
- column(id, name, scope[job/salary], filter_rule, cover)
- column_item(column_id, target_type, target_id)（专栏与招聘/薪资多对多）
- elite_program(id, company_id, name, salary_range, description, apply_url)
- salary_comment(id, report_id, user_id, content, likes, status, created_at)
- 字典表：industry, city, batch

4 API 设计总则

4.1 基础约定

- 协议：HTTPS；编码：UTF-8；数据格式：JSON。
- 路径前缀：/api/v1。
- 鉴权：HTTP Header Authorization: Bearer <jwt>。
- 命名风格：资源名小写复数（/job-postings），动作通过 HTTP 方法表达。
- 时间字段统一使用 ISO 8601 字符串。

4.2 标准方法映射

方法	HTTP	说明
Get	GET /xxx/{id}	获取单个资源
List	GET /xxx	列出资源（支持分页 / 过滤 / 关键字）
Create	POST /xxx	创建资源
Update	PATCH /xxx/{id}	部分字段更新
Replace	PUT /xxx/{id}	整体替换
Delete	DELETE /xxx/{id}	删除（软删）

4.3 分页约定

List 接口统一支持：

- page（从 1 开始，默认 1）、page_size（默认 20，最大 50）；
- 或基于游标 page_token 的方式（用于 Feed 流）。

返回结构：

```
{
  "items": [ ... ],
  "page": 1,
  "page_size": 20,
  "total": 138,
  "next_page_token": "eyJvZmZzZXQiOjIwfQ=="
}
```

4.4 统一响应结构

```
{
  "code": 0,            // 0 表示成功，其余为错误码
  "message": "ok",
  "data": { ... },
  "request_id": "9b7c..."
}
```

4.5 错误码示例

code	HTTP	含义
0	200	成功
40001	400	参数错误
40101	401	未登录 / Token 失效
40301	403	无权限
40302	403	风控拦截（疑似爬虫/作弊）
40401	404	资源不存在
42901	429	触发限流
50001	500	系统内部错误

4.6 权限控制

基于 RBAC：guest < user < member < hr < admin。每个 API 在路由表中标注最小可访问角色，并附加资源所有权约束（HR 仅能写本公司数据，普通用户仅能写本人创建的数据）。风控规则在网关层执行：黑名单用户写操作直接 403；高频访问用户触发滑块或临时封禁。

5 对外 API 详细设计

5.1 认证与用户

5.1.1 微信登录

方法 / 路径	POST `/api/v1/auth/wx-login`
权限	公开
说明	用公众号网页授权返回的 `code` 换取本系统 JWT

```
// Request
{ "code": "0c1abcDef..." }
// Response.data
{ "token": "eyJhbGc...", "token_type": "bearer", "expires_in": 7200,
  "user": { "id": 12, "role": "user" } }
```

另保留 `POST /api/v1/auth/login`（OAuth2 password flow，仅 admin 后台账号），用于 Swagger UI 内一键登录调试。

5.1.2 获取当前用户

`GET /api/v1/users/me`（需登录）

5.1.3 用户管理（管理员）

- Get: `GET /api/v1/users/{id}`
- List: `GET /api/v1/users?role=hr&q=keyword&page=1&page_size=20`
- Update: `PATCH /api/v1/users/{id}`（封禁、调整角色）
- Delete: `DELETE /api/v1/users/{id}`

5.2 公司（Company）

路径	方法	权限
/companies	GET	公开
/companies/{id}	GET	公开
/companies	POST	admin
/companies/{id}	PATCH	admin
/companies/{id}	PUT	admin
/companies/{id}	DELETE	admin
/companies:batchCreate	POST	admin

List 支持参数：`q`（公司名模糊匹配，如”字节跳动”）、`industry`、`page`、`page_size`。

5.3 招聘信息（JobPosting）——核心

5.3.1 List 招聘信息

`GET /api/v1/job-postings`

权限：公开（会员可见" 提前看" 列表）。

查询参数：

- `q`：关键字（公司/标题/岗位），模糊匹配例 `q=` 字节跳动；
- `industry`：行业 code；
- `cities`：逗号分隔，如 `cities=` 北京，上海；
- `recruitment_type`：校招/实习/社招/...；
- `batch`：招聘批次；
- `sort`：`latest|hot|deadline`（默认 latest）；
- `page`, `page_size`。

```
// Response.data.items[i]
{
  "id": 1024,
  "company": { "id": 7, "name": "极兔速递", "logo": "https://..." },
  "title": "极兔速递2026全球校园招聘启动",
  "batch": "春招正式批",
  "cities": ["上海","苏州","海外"],
  "industry": "物流/交通运输",
  "views": 1469,
  "tags": ["官方直聘","春招正式批"],
  "created_at": "2026-04-02T10:00:00+08:00"
}
```

5.3.2 Get / Create / Update / Replace / Delete

- `GET /api/v1/job-postings/{id}`（公开）
- `POST /api/v1/job-postings`（hr/admin）
- `PATCH /api/v1/job-postings/{id}`（owner-hr/admin）
- `PUT /api/v1/job-postings/{id}`（owner-hr/admin）
- `DELETE /api/v1/job-postings/{id}`（owner-hr/admin）

Create 请求体示例：

```
{
  "company_id": 7,
  "title": "极兔速递2026全球校园招聘启动",
  "batch": "春招正式批",
  "recruitment_type": "campus",
  "industry_code": "logistics",
  "cities": ["上海","苏州","海外"],
  "deliver_start": "2026-04-02",
  "deliver_end": "2026-07-31",
  "graduation_range": "2025-09-01~2026-07-31",
  "apply_url": "https://jtexpress.jobs.feishu.cn/...",
  "internal_code": "9FB8MFS",
  "benefits": ["年度奖金","带薪年假","五险一金","节日福利"],
  "description": "<p>招聘对象...</p>"
}
```

5.3.3 批量上传招聘信息

`POST /api/v1/job-postings:batchCreate`（hr/admin）

```
{ "items": [ { ...posting1 }, { ...posting2 } ] }
// Response.data
{ "success": 18, "failed": 2, "errors": [ {"index": 5, "msg": "city invalid"}
    ] }
```

另支持 `POST /api/v1/job-postings:batchUpdate`、`:batchDelete`（admin）。

5.3.4 招聘岗位（Position）子资源

- List: `GET /api/v1/job-postings/{id}/positions`
- Create: `POST /api/v1/job-postings/{id}/positions`（owner-hr）
- Update / Replace / Delete: `/api/v1/positions/{pid}`

5.4 字典（Industry / City / Batch）

- `GET /api/v1/industries`、`GET /api/v1/cities`、`GET /api/v1/batches`（公开）
- `POST/PATCH/PUT/DELETE` 仅限 admin

5.5 求职早报（DailyBrief）与订阅

- `GET /api/v1/daily-briefs?date=2026-04-27&page=1`：当日 24h 新招列表（公开）。
- `POST /api/v1/subscriptions`：用户订阅早报。
  Body: `{"type":"daily_brief"}`

- `DELETE /api/v1/subscriptions/{id}`：取消订阅。
- `GET /api/v1/subscriptions/me`：列出本人订阅。

5.6 求职好课（Course）

标准 6 方法 + List 支持 `category` 过滤、`q` 关键字检索、分页。写操作仅 admin。

5.7 求职专栏 / 薪资专栏（Column）

- Get/List/Create/Update/Replace/Delete 共用 `/api/v1/columns`，通过 `scope=job|salary` 区分。
- 专栏内容：`POST /api/v1/columns/{id}/items:batchCreate` 批量挂入；`:batchDelete` 移出。
- 列出专栏内容：`GET /api/v1/columns/{id}/items?page=1&page_size=20`。

5.8 内推集合 / 社招集合

均为 `job-postings` 之上的视图，复用 List 接口：

- 内推集合：`GET /api/v1/job-postings?has_internal_code=true`
- 社招集合：`GET /api/v1/job-postings?recruitment_type=social`

5.9 实习点评（Review）

- List 排行榜：`GET /api/v1/companies:reviewRank?tab=high|low|hot|new&q=&page=`
- Get / Create / Update / Replace / Delete：`/api/v1/reviews/{id}`（普通用户仅可操作本人记录；admin 可下架）。
- Create Body: `{"company_id":7,"score":8.5,"content":" 实习体验不错"}`

5.10 顶尖人才计划（EliteProgram）

标准 6 方法 + List 支持 `q`（公司/计划名模糊）。

5.11 薪资爆料（SalaryReport）——核心

5.11.1 List 薪资爆料

`GET /api/v1/salary-reports`

权限：公开（金额对未登录用户可能脱敏）。

查询参数：`q`（组合检索：公司/岗位/城市，例 `q= 字节跳动后端北京`）、`recruitment_type`（campus/intern/social）、`industry`、`education`、`city`、`salary_min`、`salary_max`、`sort`（latest/hot/credibility）、`page`、`page_size`。

```
// Response.data.items[i]
{
  "id": 88123,
  "company": {"id": 12, "name": "百度"},
  "position": "后端研发",
  "city": "北京",
  "salary_desc": "25k*15",
  "annual_max": 37.5,
  "education": "硕士",
  "edu_tags": ["985"],
  "credibility": 87,
  "views": 1820,
  "remark": "包含签字费 5w...",
  "tags": ["#年终奖","#签字费"],
  "created_at": "2026-04-25T12:30:00+08:00"
}
```

5.11.2 Get / Create / Update / Replace / Delete

- `GET /api/v1/salary-reports/{id}`：详情，含评论数。
- `POST /api/v1/salary-reports`：登录用户提交薪资爆料。
- `PATCH /api/v1/salary-reports/{id}`：作者本人 / admin。
- `PUT /api/v1/salary-reports/{id}`：admin（标准化字段）。
- `DELETE /api/v1/salary-reports/{id}`：作者本人 / admin。

Create 请求体：

```
{
  "company_id": 12,
  "position": "后端研发",
  "city": "北京",
  "salary_desc": "25k*15",
  "annual_min": 30,
  "annual_max": 37.5,
  "recruitment_type": "campus",
  "education": "硕士",
  "edu_tags": ["985"],
  "industry_code": "internet",
  "tags": ["#年终奖","#签字费"],
  "remark": "签字费 5w，按 15 薪折算。"
}
```

5.11.3 批量导入薪资爆料

`POST /api/v1/salary-reports:batchCreate`（admin，运营导入）

5.11.4 薪资人气榜

`GET /api/v1/salary-reports:rank?period=week|month&page=1`
返回按热度（浏览 + 点赞 + 评论加权）排序的榜单。

5.11.5 校招 / 社招视图

- 校招：`GET /api/v1/salary-reports?recruitment_type=campus&...`
- 社招：`GET /api/v1/salary-reports?recruitment_type=social&...`

5.11.6 薪资爆料评论（SalaryComment）

- List：`GET /api/v1/salary-reports/{id}/comments?page=1`
- Create：`POST /api/v1/salary-reports/{id}/comments`
- Get / Update / Replace / Delete：`/api/v1/salary-comments/{cid}`

5.12 统一搜索

`GET /api/v1/search?scope=job|salary|company&q= 字节跳动 &page=1&page_size=20`
内部走 ES，对前端暴露统一接口；返回结构同对应资源 List。

6 关键能力实现要点

6.1 部分检索（关键字）

- 直接使用 **MySQL 8.0 FULLTEXT** 索引，配合 `ngram` 分词器（`ngram_token_size=2`），可较好支持中文公司名、岗位、城市的 2-gram 子串匹配。
- 在 `job_posting(title, description)`、`salary_report(position, remark)`、`company(name, intro)` 上建 `FULLTEXT` 索引；`company.name` 同时保留 B+Tree 索引用于精确等值。
- SQL 写法：`MATCH(title) AGAINST(:q IN BOOLEAN MODE)`，结合 `IN BOOLEAN MODE` 支持 `+字节 + 跳动`这样的多 token AND。
- 跨表搜索（统一 `/search` 接口）在服务层并行三次查询并按相关度（`score = MATCH... AGAINST`）合并。
- 后期若数据量增长，可平滑切换为 Elasticsearch + ik 分词，对外 API 保持不变。

6.2 批处理

- 路径动作风格：`POST /resource:batchCreate`、`:batchUpdate`、`:batchDelete`。
- 在 FastAPI 中以独立 router 实现,请求 / 响应 schema 由 Pydantic 复用单条结构（`List[CreateIn]`）。
- 单批最多 200 条；超出在 Pydantic 层校验失败返回 422。
- 失败容忍策略：默认部分成功（返回每条结果索引与错误信息）；通过查询参数 `?atomic=true` 切换为整批 SQLAlchemy 事务，任一失败回滚。

6.3 分页

- 引入 **fastapi-pagination**：`Page[ItemOut]` 自动暴露 `page` / `size` / `total` / `pages`。
- Feed 流（薪资爆料 `/salary-reports`、早报 `/daily-briefs`）使用游标分页（`LimitOffsetPage` 或自定义 cursor），返回 `next_page_token`。
- SQL 深翻优化：`WHERE id < :last_id ORDER BY id DESC LIMIT :n`（seek-method）替代大 offset。

6.4 权限控制与反作弊

- JWT 由 `python-jose` 签发，Claims：`sub(uid)`，`role`，`company_id`，`is_blocked`，`exp`。
- FastAPI 依赖项实现：`get_current_user(token: str = Depends(oauth2_scheme))`；角色限制用 `Depends(require_role("hr"))` 装饰器化。
- 资源所有权：在 CRUD 层 `assert_owner_or_admin(obj, current_user)` 工具函数统一校验，违规抛 `HTTPException(403, 40301)`。
- 黑名单：登出 / 封禁时把 `jti` 写 Redis（TTL 与 token 同步），`get_current_user` 命中即拒绝。
- 限流：**slowapi**（基于 limits + Redis 后端）按 `(uid, ip)` 维度，公开 List 游客 60 req/min、登录用户 300 req/min；写接口默认 30 req/min。
- 反爬：未登录访问关键 List 时强制带 `captcha_token`（图形验证码 token 存 Redis，5 分钟 TTL）；UA 异常 / 高频 IP 触发滑块。
- 反作弊（薪资）：金额偏离同公司同岗位中位数 $> 3\sigma$、24h 内同账号提交 > 5 条、相同设备多账号，自动进入 `status=pending` 审核队列并降低 `credibility`；规则在 `services/risk.py` 中实现。

6.5 微信公众号能力对接（WeChatPy）

- `POST /wechat/callback`:用 `wechatpy.parse_message` 解析 XML,订阅事件写入 `subscription` 表，关键字消息转发至 `/api/v1/search` 并以图文消息回复前 5 条。

- 早报推送：APScheduler 每日 09:00 定时任务，遍历 `subscription.type=daily_brief`，调用模板消息接口；失败入 Redis 重试队列。
- 公众号 access_token：用 WeChatPy 的 `RedisStorage` 缓存，避免重复刷新。

6.6 前端（Vue 3 + Vant 4）要点

- 路由：vue-router；状态：Pinia；HTTP：axios（统一拦截器注入 `Authorization`、统一处理 `code != 0`）。
- 组件复用:`<van-search>` 顶部搜索、`<van-tabs>` 切换校招/实习/社招、`<van-list>` + `<van-pull-refresh>` 实现下拉刷新与上拉分页加载（对接 `page+page_size`）。
- OpenAPI 同步：用 **openapi-typescript** 从 `/openapi.json` 直接生成 TypeScript 类型与 axios 调用，前后端 schema 强一致。
- 视口:`<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1"` 配合 `postcss-px-to-viewport` 适配多机型。

7 需求与作业要求映射

作业要求	实现位置
标准方法 (30 分)	§5:JobPosting / SalaryReport / Company / Review / Course / Column / EliteProgram 等核心资源均提供 get/list/create/update/replace/delete
部分检索 (10 分)	§5.13 统一搜索；输入” 字节跳动” 返回招聘 / 薪资 / 公司列表
批处理 (10 分)	§5.4.3、§5.10.3：`:batchCreate / :batchUpdate / :batchDelete`
分页 (10 分)	§4.3：`page + page_size` 与 `page_token` 两种模式，全部 List 接口生效
权限控制 (10 分)	§4.6 与 §6.4：RBAC + 资源所有权 + 反作弊 + 反爬

8 后续工作

- 文档四：基于本设计完成后台代码实现，并通过微信公众号验证完整链路。
- 后续可扩展：日程同步（” 加入日程” 功能）、消息中心、数据看板（招聘/薪资统计）。